

# MILESTONE – 1 Week 1

## TRADE FINANCE WITH BLOCK CHAIN EXPLORER WITH PYTHON

### Project overview

This project is a Python backend (suitable for FastAPI or similar frameworks) organized into modular packages such as routers and services.

It uses a virtual environment, environment variable files, and a SQLite database file (app.db) along with requirements.txt to manage dependencies

### Directory structure

- .env / .env-1: Local Python virtual environments that isolate project dependencies.
- .vscode: Editor configuration for Visual Studio Code, such as settings and launch configurations.
- backend/: Main backend source folder that contains the application code and its own .env and app package.
- backend/app/: Core application package that holds routers, services, models, schemas, configs, and other backend logic.
- scripts/: Utility files such as .env and .env.example used to configure environment variables.
- app.db: Local database file, typically SQLite, used to store application data during development.
- requirements.txt / backend/requirements.txt: Lists of Python packages required by the project.

### App package layout

- app/\_\_pycache\_\_/: Auto-generated Python bytecode cache directory, not edited manually.
- app/routers/: Package containing route definitions that map URLs to request handlers; includes auth\_routes.py, docs\_routes.py, and user\_routes.py plus \_\_init\_\_.py.
- app/services/: Package containing business logic and helper functions such as authentication, configuration helpers, and database utilities.
- \_\_init\_\_.py: Marks app, routers, and services as Python packages so they can be imported elsewhere in the code.

### Core Python modules

- main.py: Application entry point that typically creates the app instance, includes routers, and configures middleware.
- auth.py: Contains authentication-related functionality such as token generation, login, and permission checks.
- config.py: Handles configuration settings, usually loading values from environment variables or configuration files.
- database.py: Manages database connections and sessions, and may define engine and session factories.

- `deps.py`: Defines dependency-injection utilities that can be reused across routers for common concerns like auth or DB access.
- `models.py`: Defines database models (ORM classes) that represent tables and relationships.
- `schemas.py`: Contains Pydantic-style schemas used for request validation and response models.

## Environment and configuration files

- `scripts/.env.example`: Template for environment variables showing which keys are required (for example, database URL and secret keys).
- `scripts/.env`: Actual environment configuration file used in development; it should not be committed publicly if it contains secrets.
- `app.db`: Database file that corresponds to the configuration in `database.py` and is used by the backend during local runs.

## How to set up and run

1. Create and activate a virtual environment (or use the existing `.venv` folder) from the project root.
2. Install dependencies with `pip install -r requirements.txt` or `pip install -r backend/requirements.txt` depending on where the app is started.
3. Copy `scripts/.env.example` to `scripts/.env` and fill in the required values such as database URL and secret keys.
4. Run the backend (for example `uvicorn app.main:app --reload` from inside backend) so that the routers and services are loaded and the API becomes accessible.

```

backend/
├── .venv/                # Virtual environment for backend
├── app/
│   ├── __pycache__/      # Python bytecode cache (auto-generated)
│   ├── routers/          # API route modules
│   │   ├── __init__.py    # Package initializer for routers
│   │   ├── auth_routes.py # Auth-related endpoints (login, register, etc.)
│   │   ├── docs_routes.py # Documentation / misc routes
│   │   └── user_routes.py  # User management endpoints
│   ├── services/         # Business logic & helpers
│   │   ├── __init__.py    # Package initializer for services
│   │   ├── auth.py        # Auth service (tokens, hashing, verification)
│   │   ├── config.py      # Configuration helpers (env, settings)
│   │   ├── database.py    # DB connection/session handling
│   │   ├── deps.py        # Dependency-injection utilities
│   │   ├── main.py        # (If present here) app startup wiring
│   │   └── models.py      # Extra service-level models/helpers
│   ├── main.py           # FastAPI entry point (creates app, includes routers)
│   ├── models.py         # ORM models (tables, relationships)
│   ├── schemas.py        # Pydantic schemas (request/response)
│   ├── database.py       # Database engine/session (if not only in services)
│   ├── config.py         # Core settings (DB URL, secrets, etc.)
│   └── deps.py           # Shared dependencies for routes
├── scripts/
│   ├── .env              # Actual environment variables (local only)
│   └── .env.example       # Sample env file for configuration
├── app.db                # Local SQLite database file
├── creation of backend... # (Optional) Notes/script file you created
└── requirements.txt       # Python dependencies for the backend

```

# MILESTONE – 1 WEEK-2

## TRADE FINANCE WITH BLOCK CHAIN EXPLORER WITH PYTHON

### Overview

The frontend is a React application created with Vite, using JSX components, React Router–style pages, and Axios for API calls.

Configuration files such as vite.config.js, tailwind.config.js, postcss.config.js, and eslint.config.js control bundling, styling, and linting.

### Directory structure

- frontend/: Root folder of the frontend project containing source, configuration, and dependency files.
- node\_modules/: Automatically generated directory where npm stores all installed packages; not edited manually.
- public/: Static files that can be served directly, such as images or icons.
- src/: Main source code directory that contains the React components, pages, styles, and API utilities.

### src folder layout

- src/api/axios.js: Central Axios instance configuration used to send HTTP requests to the backend; may include base URL, interceptors, and default headers.
- src/auth/: Authentication-related components such as:
  - Login.jsx: Login form and logic for authenticating users.
  - Register.jsx: Registration form for creating new accounts.
  - RequireAuth.jsx: Wrapper component that protects routes and redirects unauthenticated users.
- src/components/: Reusable UI components used across pages, including:
  - Layout.jsx: Main layout wrapper, often containing header, footer, and outlet for nested routes.
  - Navbar.jsx: Navigation bar component with links to different pages.
  - Protected.jsx: Higher-order or wrapper component to guard protected content.

### pages and styling

- src/pages/: Top-level page components rendered by the router:
  - Dashboard.jsx: Main dashboard view after login.
  - Documents.jsx: Page to display or manage documents.
  - Profile.jsx: User profile page.
  - Users.jsx: Page listing users or managing user data.

- Styling files:
  - App.css and index.css: Global and app-level styles that work with Tailwind and PostCSS.
  - These styles are imported into App.jsx or main.jsx to apply them across the application.

## Entry points and configs

- App.jsx: Root React component containing routes, layout, and global context providers.
- main.jsx: JavaScript entry point that renders App into the DOM, usually via ReactDOM.createRoot.
- index.html: Base HTML file used by Vite to mount the React app.
- package.json and package-lock.json: Define project metadata, dependencies, and npm scripts such as dev, build, and preview.
- vite.config.js: Vite configuration for dev server, aliases, and build options.
- tailwind.config.js and postcss.config.js: Configure Tailwind CSS and PostCSS processing for the styling pipeline.
- .gitignore and README.md: Git ignore rules and repository-level documentation for the frontend project.

structure also

```
frontend/
├─ node_modules/      # Installed npm dependencies
├─ public/             # Static assets served directly
├─ src/               # Application source code
│  ├─ api/
│  │  └─ axios.js      # Configured Axios instance (baseUrl, interceptors)
│  ├─ auth/
│  │  ├─ Login.jsx     # Login form & auth logic
│  │  ├─ Register.jsx  # User registration screen
│  │  └─ RequireAuth.jsx # Route guard for protected pages
│  ├─ components/
│  │  ├─ Layout.jsx    # Main layout (navbar + content)
│  │  ├─ Navbar.jsx    # Top navigation bar
│  │  └─ Protected.jsx # Wrapper for protected UI parts
│  ├─ pages/
│  │  ├─ Dashboard.jsx # Main dashboard page
│  │  ├─ Documents.jsx # Documents listing/management
│  │  ├─ Profile.jsx   # User profile page
│  │  └─ Users.jsx     # User list/management
│  ├─ App.css          # App-level styles
│  ├─ App.jsx          # Root React component (routes, layout)
│  ├─ index.css        # Global styles, Tailwind base
│  └─ main.jsx         # App entry; renders <App /> into DOM
├─ .gitignore          # Git ignore rules
├─ eslint.config.js    # ESLint configuration
├─ index.html          # HTML template used by Vite
├─ package-lock.json   # Exact versions of installed deps
├─ package.json        # Project metadata, scripts, dependencies
├─ postcss.config.js   # PostCSS plugins configuration
├─ README.md           # Frontend documentation
├─ tailwind.config.js  # Tailwind CSS configuration
└─ vite.config.js      # Vite bundler/dev server config
```